

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..28

REQUEST DATA

 Copy as Markdown

Request data

Once a route matches, you'll need to extract data from the request. Relic provides type-safe accessors for path parameters, query parameters, headers, and the request body.

Path parameters

Access path parameters using typed `PathParam` accessors:

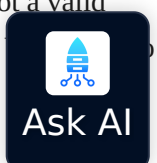
```
class UserRoute extends Route {
  static const _idParam = IntPathParam(#id);

  @override
  Future<Result> handleCall(Session session, Request request) async {
    int userId = request.pathParameters.get(_idParam);
    final user = await User.db.findById(session, userId);

    if (user == null) {
      return Response.notFound();
    }

    return Response.ok(
      body: Body.fromString(
        jsonEncode(user.toJson()),
        mimeType: MimeType.json,
      ),
    );
  }
}
```

The `IntPathParam` combines a symbol with a parser, throwing if the parameter is missing or not a valid integer. The `#id` is a Dart Symbol literal naming the `:id` segment from the route's path pattern. access raw unparsed values with `request.pathParameters.raw[#id]`.



Query parameters

Query parameters (`?key=value`) use the same typed accessor pattern:

```
class SearchRoute extends Route {
  static const _pageParam = IntQueryParam('page');

  @override
  Future<Result> handleCall(Session session, Request request) async {
    int page = request.queryParameters.get(_pageParam); // typed access
    String? query = request.queryParameters.raw['query']; // raw access
    // ...
  }
}
```

Like the path accessors, `get()` throws when the parameter is missing. Query parameters are often optional, so use raw access with a default for those, e.g.

```
int.tryParse(request.queryParameters.raw['page'] ?? '') ?? 1.
```

Headers

Access headers through `request.headers` with type-safe getters for standard HTTP headers:

```
@override
Future<Result> handleCall(Session session, Request request) async {
  // Type-safe accessors for standard headers
  final userAgent = request.headers.userAgent;
  final contentType = request.headers.contentType;
  final auth = request.headers.authorization;

  // Raw access for custom headers
  final apiKey = request.headers['X-API-Key']?.first;

  // ...
}
```

The accessors above read headers from the incoming request. To set headers on the response, pass a `headers:` value built with `Headers.build` when you construct the `Response`:

```
return Response.ok(
  body: Body.fromString('ok'),
  headers: Headers.build((h) {
    h['X-Custom-Header'] = ['value'];
  }),
);
```

Relic also exposes typed setters for standard headers. See the [Relic documentation](#) for the full header reference.

Body

Read the request body as a string for JSON or form data:

```
@override
Future<Result> handleCall(Session session, Request request) async {
  final body = await request.readAsString();
  final data = jsonDecode(body);
  // ...
}
```

WARNING

The body can only be read once. Attempting to read it again will throw a `StateError`.

Streaming bodies

For large payloads, stream instead of buffering the whole body in memory. On the request side, `request.read()` gives you the body as a stream of chunks, which suits large uploads:

```
final stream = request.read();
await for (final chunk in stream) {
  // Process chunk
}
```

On the response side, `Body.fromDataStream` streams data to the client as it is produced, for example when serving a large file:

```
Stream<Uint8List> dataStream = getFileStream();

return Response.ok(
  body: Body.fromDataStream(
    dataStream,
    contentLength: fileSize,
    mimeType: MimeType.octetStream,
  ),
);
```

For more details on typed parameters and headers, see the [Relic documentation](#).

Next steps

- **Middleware** - Intercept and transform requests and responses
- **Static files** - Serve static assets
- **Server-side HTML** - Render HTML on the server